Grid Lights / Selectable Cells (Drag-Select)

Problem: Build (1) the classic *Grid Lights* game: click cells to light them up; once every cell is lit, they switch off one by one in the **reverse order** they were lit. (2) A *selectable grid*: press on a cell and drag to select a rectangle of cells.

Often asked at: Uber (grid lights), Meta (drag-select)

Tech stack: React (Vite, JavaScript, Tailwind CSS)

Time limit: 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements**: restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List interactions / use cases**: every user action and what must happen. This is your test checklist.
3. **Design the architecture**: components and their single responsibility.
4. **Design the state**: the minimum state; derive everything else.
5. **Design data structures and contracts**: shapes of data and component props.
6. **Trace one data flow end to end** before coding, to catch design holes.
7. **Find the hard parts** early and decide the solution before typing.
8. **Build in layers**: simplest working version first, then add one working layer at a time.
9. **Cover states and edge cases.**
10. **Finalize structure, code, and wrap up** with trade-offs and next steps.

Time budget (60 minutes)

Phase	Time
Requirements, questions, assumptions	5 min
Use cases, architecture, state design	7 min
Grid Lights: render grid + click to light	8 min
Grid Lights: reverse deactivation with timers + lock	10 min
Drag-select: pointer down / move / up + derived rectangle	15 min
Edge cases, accessibility, polish	8 min
Walkthrough, trade-offs, questions	7 min

Think out loud

Say every decision before you make it: *"I'll store the activation order as an array, because 'reverse order' means a stack: I push on click and pop while switching off."* The interviewer grades how you reason, not just whether the cells turn green.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

Grid Lights

- Render a grid from a config matrix, e.g. 3×3 where the centre cell is empty (`0` = no cell).
- Clicking an unlit cell lights it (green).
- When **all** cells are lit, they switch off **one at a time, every 300 ms, in the reverse order they were lit**.
- While switching off, clicks are ignored.

Selectable cells (drag-select)

- Render an `R × C` grid.
- Press on a cell, drag, release: every cell inside the rectangle from the start cell to the current cell is selected.
- Selection updates live while dragging, in any direction (up-left as well as down-right).
- Show how many cells are selected and allow clearing.

Non-functional requirements

Area	Requirement
Performance	Re-render only when the pointer enters a *new* cell, not on every pixel of movement.
Correctness	Timers are always cleaned up: no state updates after unmount, no double timers.
Error handling	Releasing the mouse outside the grid (or outside the window) must end the drag.
Accessibility	Light cells are real `<button>`s with `aria-pressed`; selection grid uses `role="grid"` + `aria-selected`; count announced with `aria-live`.
Usability	No native text selection while dragging; works with mouse **and** touch.
Reusability	Both components take props (`config`, `delay`, `rows`, `cols`).
Extensibility	Pure geometry helpers (`getBounds`, `isInBounds`) can be reused for multi-rectangle or keyboard selection.

Clarifying questions

1. Can a lit cell be clicked again to switch it off, or is a click on a lit cell ignored? *(Assume ignored.)*
2. Should the switch-off start immediately after the last click, or after one delay? *(Assume after one delay, then one per delay.)*
3. Are clicks allowed during switch-off? *(Assume no: grid is locked.)*
4. For drag-select: one rectangle, or many (Ctrl/Shift to add)? *(Assume one rectangle; new drag replaces it.)*
5. Must it work on touch devices? *(Assume yes: use Pointer Events.)*
6. Does a single click (no drag) select one cell? *(Assume yes.)*

Assumptions

- Grid Lights config is a 2D array of `1` / `0`; default 3×3 with an empty centre; delay 300 ms.
- A cell is identified by its flat index `row * cols + col`.
- Drag-select keeps one rectangle, stored as two corners: **anchor** (where the drag started) and **focus** (where the pointer is now).
- Mouse and touch both supported; keyboard selection is a stretch goal.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page load	Both grids render; no cell lit; no cell selected.
Click an unlit light	Cell turns green; its index is pushed to `order`.
Click an already lit light	Nothing happens.
Click the empty (centre) slot	Nothing: it is not a button.
Light the last unlit cell	Grid locks; after 300 ms the last-lit cell switches off, then the next, every 300 ms.
Click during switch-off	Ignored (buttons disabled).
Last cell switches off	Grid unlocks; user can play again.
Component unmounts mid switch-off	Pending timer is cleared; no warnings.
Pointer down on a selectable cell	That one cell is selected; drag starts.
Drag to another cell	Rectangle between start and current cell is selected (any direction).
Drag outside the grid	Selection stays at the last cell inside the grid.
Release pointer (inside or outside the grid)	Drag ends; selection stays.
Release mouse outside the browser window	Next move with no buttons pressed ends the drag.
Drag with a finger on touch screen	Same as mouse; page does not scroll.

Action / input	Expected behavior / output
Click "Clear"	Selection removed; status text resets.

3. Design the architecture

Module	Responsibility
`App`	Page layout; renders both demos.
`GridLights`	Owns activation order + lock; renders lights; runs the reverse switch-off timer.
`SelectableGrid`	Owns anchor/focus/dragging; handles pointer events; renders cells and selection status.
`utils/grid.js`	Pure geometry: normalize two corners into bounds, test a cell, count cells.

```
App
├── GridLights           (state: order[], deactivating)
│   └── <button> × N     (lit = order.includes(i))
└── SelectableGrid       (state: anchor, focus, dragging)
    ├── <div role=grid>  (one pointerdown + one pointermove listener)
    │   └── gridcell × R*C  (selected = isInBounds(bounds, r, c))
    └── status + Clear

          ▲
          └── utils/grid.js  (getBounds, isInBounds, countInBounds)
```

Start simple: no context, no reducer, no separate `Cell` component. Each grid's state is local because nothing else needs it. Split a `Cell` out only if it needs `memo` for big grids.

4. Design the state / data model

Grid Lights

State	Purpose
`order: number[]`	Indices of lit cells **in the order they were lit**. A stack.
`deactivating: boolean`	True while switching off; locks clicks and drives the timer.

Selectable Grid

State	Purpose
`anchor: {r, c} | null`	Cell where the drag started.
`focus: {r, c} | null`	Cell the pointer is currently over.

State	Purpose
`dragging: boolean`	Whether pointer moves should update `focus`.

```
// GridLights
const [order, setOrder] = useState([]);
const [deactivating, setDeactivating] = useState(false);

// SelectableGrid
const [anchor, setAnchor] = useState(null);
const [focus, setFocus] = useState(null);
const [dragging, setDragging] = useState(false);
```

Senior note: derive, don't store

- `lit` **per cell** is derived: `order.includes(index)`. Storing a separate `lit[]` boolean array next to `order` creates two sources of truth that can disagree.
- `allLit` is `order.length === total`; it is checked in the click handler, not stored.
- **The selected rectangle** and **the selected count** are derived from `anchor` + `focus` on every render. Storing a `Set` of selected cells means rebuilding it on every move and risking stale cells when the user drags back.
- Ownership: each component owns its own state; `App` holds nothing. If a parent needs the selection, lift `anchor/focus` up or add an `onChange(bounds)` prop.

---

5. Design the data structures and contracts

Example data

```
// Grid Lights config: 1 = cell, 0 = empty slot
const config = [
  [1, 1, 1],
  [1, 0, 1],
  [1, 1, 1],
];
// flat: [1,1,1,1,0,1,1,1,1]  -> 8 real cells, index 4 is empty

// After clicking cells 0, 8, 2:
order = [0, 8, 2];      // switch-off sequence will be 2, 8, 0

// Drag from (row 3, col 4) up-left to (row 1, col 1)
anchor = { r: 3, c: 4 };
focus  = { r: 1, c: 1 };
bounds = { top: 1, bottom: 3, left: 1, right: 4 }; // 3 × 4 = 12 cells
```

Why the **flat index** as the id: the grid never reorders, so the position is a stable, unique key and fits nicely in an array. Why **row/col** for selection: rectangle math is about rows and columns, so `{r, c}` avoids dividing and modding on every check.

Component contracts

Component	Props	Output
`GridLights`	`config?: number[][]` (default 3×3 ring), `delay?: number` (ms, default 300)	Renders lights; no callbacks needed.
`SelectableGrid`	`rows?: number` (default 8), `cols?: number` (default 10)	Renders grid + status. Easy extension: `onChange(bounds)`.

Utility contracts

```
getBounds(a, b)           // ({r,c}|null, {r,c}|null) -> {top,bottom,left,right} | null
isInBounds(bounds, r, c)  // -> boolean (false when bounds is null)
countInBounds(bounds)     // -> number of cells in the rectangle (0 when null)
```

No API: everything is client-side.

---

6. Trace the data flow

Grid Lights: last click to full switch-off

```
User clicks the last unlit cell (index 6)
        |
        ▼
activate(6): not locked, not already lit
        |
        ▼
next = [...order, 6]  -> length === total (8)
setOrder(next); setDeactivating(true)
        |
        ▼
Render: all 8 green, buttons disabled
        |
        ▼
useEffect([deactivating, order]) schedules setTimeout(300ms)
        |
        ▼
Timer fires: setOrder(prev => prev.slice(0, -1))   (cell 6 switches off)
        |
        ▼
order changed -> cleanup clears old timer -> effect schedules the next one
        |   ... repeats, popping one index per 300 ms ...
        ▼
Last timer: order.length was 1 -> setOrder([]) + setDeactivating(false)
        |
        ▼
Render: all cells off, buttons enabled again
```

Drag-select: one drag

```
pointerdown on cell (3,4)
  -> cellFromPoint(e) reads data-r / data-c
  -> anchor = focus = {3,4}; dragging = true
  -> effect adds window 'pointerup' listener
        |
pointermove over the grid
  -> cellFromPoint(e) = {2,3}; differs from focus -> setFocus({2,3})
  -> render: bounds = getBounds(anchor, focus); each cell checks isInBounds
        |
pointerup anywhere on the page
  -> window listener: dragging = false; listener removed
  -> anchor/focus stay -> selection stays on screen
```

---

7. Important technical challenges

Challenge 1: "Reverse order" switch-off

Problem: User lights cells 0 → 8 → 2. Switch-off must go 2 → 8 → 0. If you store lit cells as a boolean array or a `Set`, the order is lost and you cannot replay it backwards.

Solution: Store the activation order as an array and treat it as a **stack**: push on click, pop on each tick. "Lit" is derived as `order.includes(i)`.

```
const activate = (index) => {
  if (deactivating || order.includes(index)) return;
  const next = [...order, index];
  setOrder(next);
  if (next.length === total) setDeactivating(true);
};
```

Challenge 2: Timers that don't leak, double-fire or go stale

Problem: A `setInterval` that reads `order` from its closure keeps seeing the *old* array, so it pops the same item forever. Without cleanup, unmounting mid-sequence calls `setState` on a dead component, and React StrictMode (which mounts effects twice in dev) would start two intervals that switch off two cells per tick.

Solution: One `setTimeout` per step inside an effect that depends on `order`. Each pop changes `order`, React runs the cleanup (clearing any pending timer) and schedules the next step. Use a functional update so the pop always uses the latest state.

```
useEffect(() => {
  if (!deactivating) return;
  const id = setTimeout(() => {
    setOrder((prev) => prev.slice(0, -1));
    if (order.length === 1) setDeactivating(false);
  }, delay);
  return () => clearTimeout(id);
}, [deactivating, order, delay]);
```

Challenge 3: Drag direction and rectangle math

Problem: User drags from (3,4) **up-left** to (1,1). A naive check `r >= anchor.r && r <= focus.r` is false for every row, so nothing gets selected.

Solution: Normalize the two corners with `min` / `max` into `{top, bottom, left, right}`. Selection is then a pure function of two points, so dragging back shrinks the rectangle for free.

```
export function getBounds(a, b) {
  if (!a || !b) return null;
  return {
    top: Math.min(a.r, b.r), bottom: Math.max(a.r, b.r),
    left: Math.min(a.c, b.c), right: Math.max(a.c, b.c),
  };
}
```

Challenge 4: Pointer events that work everywhere

Problem: Three real-world breakages:

1. With `onMouseEnter` per cell, **touch** never works: during a touch drag the browser does not fire enter/leave on other elements (the finger is captured by the first one).
2. Release the mouse **outside the grid** and `onMouseUp` on the grid never fires, so the grid stays in "dragging" mode and keeps selecting on hover.
3. Dragging selects page text and, on touch, scrolls the page.

Solution:

- Use **Pointer Events** on the grid container and find the cell under the pointer with `document.elementFromPoint` + `data-r/data-c`. One listener instead of `R × C` listeners, and it works for mouse, pen and touch.
- Listen for `pointerup` / `pointercancel` on `window` while dragging; if the mouse was released outside the browser window, the next move has `buttons === 0`, so end the drag then.
- `e.preventDefault()` on pointerdown, `select-none` and `touch-action: none` (`touch-none`) on the grid.

```
function cellFromPoint(e) {
  const el = document.elementFromPoint(e.clientX, e.clientY)?.closest('[data-cell]');
  return el ? { r: Number(el.dataset.r), c: Number(el.dataset.c) } : null;
}

const handlePointerMove = (e) => {
  if (!dragging) return;
  if (e.pointerType === 'mouse' && e.buttons === 0) return setDragging(false);
  const cell = cellFromPoint(e);
  if (cell && (cell.r !== focus.r || cell.c !== focus.c)) setFocus(cell); // only on a new cell
};
```

8. Implement in the correct order

#	Step	Working result
1	Vite + React + Tailwind; `App` with two sections.	Page renders.
2	`GridLights`: render cells from `config` with CSS grid; skip `0` slots.	Static 3×3 ring.
3	`order` state + `activate`; colour = `order.includes(i)`.	Clicking lights cells.
4	Detect "all lit" in `activate`; set `deactivating`; disable buttons.	Grid locks when full.
5	Timer effect popping one index per `delay` with cleanup.	Reverse switch-off works.
6	`utils/grid.js`: `getBounds`, `isInBounds`, `countInBounds` + a quick assert.	Geometry proven.
7	`SelectableGrid`: render `rows × cols` cells with `data-r/data-c`.	Static grid.
8	Pointer down / move on the container; anchor + focus; derived selection.	Drag selects a rectangle.
9	Window `pointerup` / `pointercancel`; `buttons === 0` guard.	Drag always ends.
10	`select-none`, `touch-none`, `preventDefault`; status text + Clear.	Clean UX on mouse and touch.
11	ARIA: `aria-pressed`, `aria-label`, `role=grid`, `aria-selected`, `aria-live`.	Accessible.

9. Cover all states and edge cases

State / edge case	Handling
Initial	`order = []`, no selection, helper text "Drag across the grid…".
Playing (some lit)	Lit cells green; unlit clickable.
All lit / deactivating	Buttons disabled; one cell off per `delay`.

State / edge case	Handling
Finished	`order = []`, `deactivating = false`; can replay.
Click lit cell	Ignored (`order.includes`).
Empty slot (`0`)	Rendered as an inert `<span aria-hidden>`, keeps grid shape.
Config with all zeros	`total = 0`, nothing to click, no timer ever starts.
Unmount during switch-off	Effect cleanup clears the timeout.
StrictMode double effects	Each run's cleanup clears its timer; only one timer is live.
`delay` prop changes mid-sequence	Effect re-runs with the new delay.
Single click (no drag)	1×1 selection.
Drag in any direction	`min/max` normalization.
Drag back toward anchor	Rectangle shrinks (derived from two points).
Pointer leaves grid	`cellFromPoint` returns `null`; focus stays at last cell.
Release outside grid	Window `pointerup` ends the drag.
Release outside window	`buttons === 0` on next move ends the drag.
Right / middle click	Ignored (`e.button !== 0`).
Touch drag	Pointer events + `touch-action: none`.
Gap between cells	`elementFromPoint` hits the container, `closest('[data-cell]')` is null → ignored.
Clear	Resets anchor and focus.

10. Final structure

```
_65_grid_lights_selectable_cells/
├── index.html
├── package.json
├── vite.config.js            (react + @tailwindcss/vite plugins)
└── src/
    ├── main.jsx
    ├── index.css             (@import "tailwindcss")
    ├── App.jsx
    ├── utils/
    │   └── grid.js           (getBounds, isInBounds, countInBounds)
    └── components/
        ├── GridLights.jsx
        └── SelectableGrid.jsx
```

11. Full working code

`src/utils/grid.js`

```
// Normalize two corner cells (in any drag direction) into a rectangle.
export function getBounds(a, b) {
  if (!a || !b) return null;
  return {
    top: Math.min(a.r, b.r),
    bottom: Math.max(a.r, b.r),
    left: Math.min(a.c, b.c),
    right: Math.max(a.c, b.c),
  };
}

export function isInBounds(bounds, r, c) {
  return (
    !!bounds &&
    r >= bounds.top &&
    r <= bounds.bottom &&
    c >= bounds.left &&
    c <= bounds.right
  );
}

export function countInBounds(bounds) {
  if (!bounds) return 0;
  return (bounds.bottom - bounds.top + 1) * (bounds.right - bounds.left + 1);
}
```

`src/components/GridLights.jsx`

```
import { useEffect, useState } from 'react';

const DEFAULT_CONFIG = [
  [1, 1, 1],
  [1, 0, 1],
  [1, 1, 1],
];

export default function GridLights({ config = DEFAULT_CONFIG, delay = 300 }) {
  const cells = config.flat();
  const cols = config[0].length;
  const total = cells.filter(Boolean).length;

  // Activation order is a stack: push on click, pop while deactivating.
  const [order, setOrder] = useState([]);
  const [deactivating, setDeactivating] = useState(false);

  useEffect(() => {
    if (!deactivating) return;
    // One timer per step; cleanup clears it on unmount or before the next step.
    const id = setTimeout(() => {
      setOrder((prev) => prev.slice(0, -1));
      if (order.length === 1) setDeactivating(false);
    }, delay);
    return () => clearTimeout(id);
  }, [deactivating, order, delay]);

  const activate = (index) => {
    if (deactivating || order.includes(index)) return;
    const next = [...order, index];
    setOrder(next);
    if (next.length === total) setDeactivating(true);
  };

  return (
    <div
      className="grid gap-2 w-fit"
      style={{ gridTemplateColumns: `repeat(${cols}, 4rem)` }}
    >
      {cells.map((visible, index) => {
        if (!visible) return <span key={index} aria-hidden="true" />;
        const lit = order.includes(index);
        return (
          <button
            key={index}
```

```
            type="button"
            aria-label={`Row ${Math.floor(index / cols) + 1}, column ${(index % cols) + 1}`}
            aria-pressed={lit}
            disabled={deactivating}
            onClick={() => activate(index)}
            className={`h-16 w-16 rounded border-2 border-slate-700 transition-colors
              focus-visible:outline-3 focus-visible:outline-blue-500
              ${lit ? 'bg-green-500' : 'bg-white hover:bg-slate-100'}
              disabled:cursor-not-allowed`}
          />
        );
      })}
    </div>
  );
}
```

`src/components/SelectableGrid.jsx`

```
import { useEffect, useState } from 'react';
import { countInBounds, getBounds, isInBounds } from '../utils/grid';

// Find the cell under the pointer. Works for mouse and touch
// (touch never fires enter/leave on other cells while the finger is down).
function cellFromPoint(e) {
  const el = document.elementFromPoint(e.clientX, e.clientY)?.closest('[data-cell]');
  return el ? { r: Number(el.dataset.r), c: Number(el.dataset.c) } : null;
}

export default function SelectableGrid({ rows = 8, cols = 10 }) {
  const [anchor, setAnchor] = useState(null); // where the drag started
  const [focus, setFocus] = useState(null); // where the pointer is now
  const [dragging, setDragging] = useState(false);

  const bounds = getBounds(anchor, focus); // derived, never stored
  const count = countInBounds(bounds);

  // Pointer can be released outside the grid: listen on window.
  useEffect(() => {
    if (!dragging) return;
    const stop = () => setDragging(false);
    window.addEventListener('pointerup', stop);
    window.addEventListener('pointercancel', stop);
    return () => {
      window.removeEventListener('pointerup', stop);
      window.removeEventListener('pointercancel', stop);
    };
  }, [dragging]);

  const handlePointerDown = (e) => {
    const cell = cellFromPoint(e);
    if (!cell || e.button !== 0) return;
    e.preventDefault(); // no native text selection while dragging
    setAnchor(cell);
    setFocus(cell);
    setDragging(true);
  };

  const handlePointerMove = (e) => {
    if (!dragging) return;
    // Mouse released outside the browser window: no pointerup ever came.
    if (e.pointerType === 'mouse' && e.buttons === 0) return setDragging(false);
    const cell = cellFromPoint(e);
    // Only update when the pointer crosses into a new cell.
```

```
  if (cell && (cell.r !== focus.r || cell.c !== focus.c)) setFocus(cell);
};

const clear = () => {
  setAnchor(null);
  setFocus(null);
};

return (
  <div className="space-y-3">
    <div
      role="grid"
      aria-multiselectable="true"
      aria-label="Selectable cells"
      onPointerDown={handlePointerDown}
      onPointerMove={handlePointerMove}
      className="grid gap-1 w-fit select-none touch-none"
      style={{ gridTemplateColumns: `repeat(${cols}, 2.5rem)` }}
    >
      {Array.from({ length: rows * cols }, (_, i) => {
        const r = Math.floor(i / cols);
        const c = i % cols;
        const selected = isInBounds(bounds, r, c);
        return (
          <div
            key={i}
            role="gridcell"
            aria-selected={selected}
            data-cell=""
            data-r={r}
            data-c={c}
            className={`h-10 w-10 rounded border border-slate-300 ${
              selected ? 'bg-blue-500 border-blue-600' : 'bg-white'
            }`}
          />
        );
      })}
    </div>

    <div className="flex items-center gap-3 text-sm" aria-live="polite">
      <span>
        {count === 0
          ? 'Drag across the grid to select cells'
          : `${count} cell${count > 1 ? 's' : ''} selected (${
              bounds.bottom - bounds.top + 1
            } × ${bounds.right - bounds.left + 1})`}
      </span>
```

```
        {count > 0 && (
          <button
            type="button"
            onClick={clear}
            className="rounded border px-2 py-1 hover:bg-slate-100"
          >
            Clear
          </button>
        )}
      </div>
    </div>
  );
}
```

src/App.jsx

```
import GridLights from './components/GridLights';
import SelectableGrid from './components/SelectableGrid';

export default function App() {
  return (
    <main className="min-h-screen p-8 space-y-10 bg-slate-50 text-slate-900">
      <section className="space-y-3">
        <h1 className="text-2xl font-bold">Grid Lights</h1>
        <p className="text-sm text-slate-600">
          Click every cell. When all are lit they switch off in reverse order.
        </p>
        <GridLights />
      </section>

      <section className="space-y-3">
        <h2 className="text-2xl font-bold">Selectable Cells</h2>
        <SelectableGrid rows={8} cols={10} />
      </section>
    </main>
  );
}
```

`src/main.jsx`

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

`src/index.css`

```
@import "tailwindcss";
```

Quick self-check for the geometry (run with `node`)

```
import assert from 'node:assert';
import { getBounds, isInBounds, countInBounds } from './src/utils/grid.js';

const b = getBounds({ r: 3, c: 4 }, { r: 1, c: 1 }); // dragged up-left
assert.deepEqual(b, { top: 1, bottom: 3, left: 1, right: 4 });
assert.equal(countInBounds(b), 12);
assert(isInBounds(b, 2, 2));
assert(!isInBounds(b, 0, 2));
assert.equal(countInBounds(getBounds({ r: 2, c: 2 }, { r: 2, c: 2 })), 1);
console.log('grid utils ok');
```

---

12. Wrap-up

Edge cases handled

- Reverse-order switch-off driven by a stack, not a boolean array.
- Grid locked during switch-off; lit cells ignore clicks; empty slots are inert.
- Timer cleanup on unmount and between steps; safe under StrictMode.
- Drag in any direction, drag back to shrink, single-click select.
- Drag ends on release anywhere, even outside the browser window.
- Mouse, pen and touch; no text selection or page scroll while dragging.
- Accessible buttons/grid cells with ARIA state and a live status message.

Trade-offs to mention

Decision	Why	Cost
`order.includes(i)` per cell	Single source of truth, simple.	O(n) per cell → O(n^2) render; fine for 8 cells. For big grids use a `Set` derived with `useMemo`.
One `setTimeout` per step (effect on `order`)	No stale closures, trivial cleanup.	Re-runs the effect each step (cheap).
Selection stored as two corners	Tiny state, derived rectangle, shrink for free.	Only one rectangle; multi-select needs a list of rectangles.
`elementFromPoint` on the container	One listener, works for touch.	A DOM query per move; fine, and we only `setState` on a new cell.
No `Cell` component / no `memo`	Less code for 80 cells.	For 10k cells, every move re-renders all cells.

What to add with more time

- **Tests:** React Testing Library: click order → assert reverse switch-off with `vi.useFakeTimers()`; drag with `fireEvent.pointerDown/Move/Up` (mock `elementFromPoint`).
- **Keyboard:** arrow keys to move focus, Space to light / select, Shift+Arrow to extend the selection (roving `tabIndex`).
- **Multi-select:** Ctrl/Cmd-drag adds rectangles; store `rectangles[]`.
- **Performance for huge grids:** memoized `Cell` that only re-renders when its `selected` flag changes, or paint the selection as one absolutely positioned overlay rectangle instead of colouring each cell; virtualize rows.
- **Customization:** `onChange(bounds)` / `onComplete()` callbacks, configurable colours, "restart" button.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why store the activation order instead of a boolean per cell?

Because the requirement is "switch off in reverse order". A boolean array tells you *which* cells are on but not *when* they were lit. An ordered array is a stack: push on click, pop on each tick. Booleans are derived from it.

Q2. What if the interviewer says clicking a lit cell should switch it off?

Remove it from the stack: `setOrder(order.filter(i => i !== index))`. The reverse sequence still works because the remaining order is preserved. Only the all-lit check needs no change.

Q3. What if the grid config comes from an API or changes at runtime?

`total` and `cols` are derived from `config` on every render, so a new config works. Reset `order` when the config changes (pass `key={configId}` to remount, the simplest way), otherwise old indices could point at

missing cells.

State design

Q4. Why not keep `deactivating` derived instead of stored?

It can't be derived from `order` alone: `order.length === total` is true only at the start of switch-off; during the sequence the length shrinks but the grid must stay locked. So it is a real piece of state: the "mode".

Q5. Why is the selection derived instead of stored as a `Set` of cells?

Two corners fully describe a rectangle. A stored `Set` must be rebuilt on every move and goes stale if you forget to remove cells when the user drags back. Deriving with `getBounds` is O(1) state and always consistent.

Q6. Would you use `useReducer` here?

For two or three fields, `useState` is clearer. If more modes come in (paused, restarting, multi-select with modifiers), a reducer with explicit actions (`CLICK` , `TICK` , `RESET`) makes transitions testable without rendering.

Challenge: timers

Q7. Why not a single `setInterval` ?

It works only with a functional update and a ref to stop it, and its closure captures the first `order`, which is easy to get wrong. One `setTimeout` per step keyed on `order` always sees fresh state and has a trivial cleanup. Both are acceptable if you explain the stale-closure risk.

Q8. What happens in React StrictMode in development?

Effects mount, clean up and mount again. Because the cleanup clears the timeout, only one timer is alive. With a missing cleanup you'd see two cells switch off per tick: a common interview bug.

Q9. What if the user navigates away mid switch-off?

The component unmounts, React runs the cleanup, `clearTimeout` cancels the pending step. No state update on an unmounted component.

Challenge: drag-select events

Q10. Why `elementFromPoint` instead of `onMouseEnter` on each cell?

`onMouseEnter` per cell does not fire during a touch drag (the touch is captured by the first element), and it adds `R × C` handlers. One `pointermove` on the container + `elementFromPoint` handles mouse, pen and touch with a single listener (event delegation).

Q11. Why listen to `pointerup` on `window`?

The user often releases outside the grid. A grid-level `onPointerUp` would miss it and leave the component stuck in dragging mode. The listener is added only while dragging and removed in the effect cleanup.

Q12. Why not `setPointerCapture`?

Capture sends all moves to the captured element, which is great for knowing about the pointer, but you still have to find the cell under it, so `elementFromPoint` is needed anyway. It's a valid alternative to the window listener for ending the drag.

Q13. How do you stop text selection and page scrolling while dragging?

`e.preventDefault()` on `pointerdown`, `user-select: none` (`select-none`) and `touch-action: none` (`touch-none`) on the grid. Without `touch-action`, the browser treats a finger drag as a scroll and fires `pointercancel`.

Performance and scaling

Q14. What if the selectable grid is 1000 × 1000?

Rendering 1M DOM cells is the problem, not the math. Virtualize rows/columns (render only visible cells), compute the hovered cell from coordinates (`Math.floor((x - left) / cellSize)`) instead of `elementFromPoint`, and draw the selection as a single overlay `div` (or on a `<canvas>`) instead of re-colouring every cell.

Q15. Every pointer move triggers a render?

No. We call `setFocus` only when the pointer enters a **different** cell, so a drag across 10 cells causes about 10 renders, not hundreds. For more, throttle with `requestAnimationFrame`.

Q16. `order.includes(i)` is O(n) per cell: is that a problem?

For 8 cells, no. For a large light grid, derive `const litSet = useMemo(() => new Set(order), [order])` and use `litSet.has(i)` for O(1) lookups.

Accessibility and trade-offs

Q17. How would a keyboard user play Grid Lights?

Cells are native `<button>`s, so Tab + Enter/Space already work. `aria-pressed` announces lit state and `aria-label` gives the row/column. While locked, `disabled` tells assistive tech the grid is busy.

Q18. How would you add keyboard selection to the drag grid?

Roving `tabIndex` on cells, arrow keys move the focus cell, Space sets the anchor, Shift+Arrow moves `focus` while keeping `anchor`: the same `anchor/focus` state and `getBounds` work unchanged. That's why the selection model is two corners, like the browser's own text selection.

Q19. Why not a library like `react-selecto` or `@dnd-kit`?

In an interview the point is to show event handling. In production, a library is fine for complex cases (multi-select, scroll containers, auto-scroll), but for one rectangle in a fixed grid it adds weight for about 40 lines of code.

Q20. How would you test the reverse order?

With fake timers: click cells 0, 8, 2, …, last; assert all `aria-pressed="true"`; `vi.advanceTimersByTime(300)` and assert the last-clicked cell is off; advance again and check the previous one; at the end all buttons are enabled again.

```
vi.useFakeTimers();
clicks.forEach((i) => fireEvent.click(cells[i]));
act(() => vi.advanceTimersByTime(300));
expect(cells[clicks.at(-1)]).toHaveAttribute('aria-pressed', 'false');
```